BÖLÜM 9 / MODULE 9

Değişiklik Talepleri, Hatalar (Bugs) ve Mevcut (Brownfield) Sistemler

Change Requests, Bugs, and Brownfield Systems

15. İlke: 'Tüm İşler Bir Use Case Değişikliğidir'. İmza testi (Sign-off test) ile hata vs geliştirme ayrımı, neden /bug komutunun olmadığı, mevcut (brownfield) sistemlerin niyetini tersine mühendislikle kurtarma, 40 REST ucunu 4-8 use case'e indirgeme ve SE-RE ortak taban çizgisi (baseline) incelemesi.

Principle 15: 'All Work is a Change to a Use Case'. The sign-off test to classify bugs vs enhancements, why there is no /bug command, recovering specifications from legacy brownfield systems, collapsing 40 REST endpoints into 4-8 use cases, and the joint SE-RE baseline review.

49

BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

15. İlke: Tüm İşler Bir Use Case Değişikliğidir (Özellik, Değişiklik, Hata)

Principle 15: All Work is a Change to a Use Case (Features, Changes, and Bugs)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              15. İLKE: HER İŞ BİR USE CASE DEĞİŞİKLİĞİDİR    │
└──────────────────────────────────────────────────────────────┘

Masaüstünüze Gelen İş Türleri:

1. YENİ ÖZELLİK (New Feature) ──► Sisteme YENİ BİR USE CASE ekler!
   - Gereksinim kataloğunda (requirements.md) yeni bir kullanıcı hikayesi açar.

2. DEĞİŞİKLİK TALEBİ (Change)  ──► MEVCUT BİR USE CASE'İ günceller!
   - Bir iş kuralı (BR) değişir, yeni bir alternatif akış (A4) eklenir.

3. YAZILIM HATASI (Bug Report) ──► KOD İLE ŞARTNAME ARASINDAKİ ÇELİŞKİDİR!
   - Ya kod şartnameden sapmıştır (kod düzeltilir).
   - Ya da şartname eksik kalmıştır (şartname düzeltilir ve kod yenilenir).

🎯 MİMARİ İLKE:
"Hiçbir kod değişikliği, şartnameden geçmeden sisteme GİREMEZ!
 Aksi halde şartname çürümeye başlar ve kod tekrar tek gerçek haline gelir!"
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphaneyi düşünün: Kütüphaneye yeni bir kitaplık eklemek (Yeni Özellik), mevcut kitaplığın sırasını değiştirmek (Değişiklik) veya yanlış rafa konmuş bir kitabı doğru rafa koymak (Hata)... Hepsi kütüphane kataloğuna (Use Case Şartnamesi) işlenmek zorundadır! Eğer rafa yeni kitap koyup kataloğa yazmazsanız, o kütüphane 1 hafta sonra çöplüğe döner!

🇬🇧 ENGLISH (ELI5)

Think of a grand library: Adding a new science section (New Feature), rearranging the history shelves (Change Request), or reshelving a misplaced book (Bug fix)... All three require updating the master library catalog (Use Case Spec)! If you move books without updating the catalog, the library descends into chaos within a week.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

15. İlke: Tüm İşler Bir Use Case Değişikliğidir (All Work is a Change to a Use Case): Yazılım projelerinde işler geliştiricilerin masasına tertemiz sıfır kilometre use case'ler halinde gelmez; Pazartesi bir değişiklik talebi, Salı günü ise bir bug raporu olarak düşer. AI Unified Process'te her üç iş türü de temelde aynı şeydir: Bir use case değişikliği! 1) Yeni Özellik: Sisteme yeni bir use case (UC-XXX) ekler veya mevcut olanı yeni aktör ve akışlarla genişletir. 2) Değişiklik Talebi: Var olan use case'i revize eder; örneğin 28 günlük ödünç süresini 14 güne düşürür (BR-004 güncellemesi). 3) Bug: Şartname ile kod arasındaki uyuşmazlıktır. Çoğunlukla kod yanlıştır; bazen de şartname paydaşın gerçek iradesini eksik yansıtmıştır. Çözüm her halükarda önce şartnamenin incelenmesi ve güncellenmesiyle başlar. Bu disiplin korunmazsa 'Principle 11' (Şartname koddan daha uzun yaşar) anında çöker.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 15: All Work is a Change to a Use Case: Software engineering reality rarely delivers pristine greenfield specifications. Work arrives as frantic Jira tickets, customer change requests, or production incidents. In the AI Unified Process, all three are structurally identical: a change to a use case specification. 1) New Features add use case specifications or attach new actors/flows. 2) Change Requests modify existing preconditions, business rules, or alternative flows. 3) Bugs represent disagreements between specification and reality. If a side-channel exists where developers patch production code directly without updating specifications, the specification rots and code reverts to being the fragile truth. By enforcing that every change flows specification-first, the architecture retains continuous regenerability.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Tüm İş Değişikliklerinin Şartname Tabanlı Mühürlenmesi
package com.maang.aiup.allwork;

public class WorkDeltaArchitecture {
    // İş Kuralı BR-004 Değişikliği: 28 günden 14 güne indirildi
    public record LoanPolicy(int maxDaysAllowed) {
        // Eski sürüm: maxDaysAllowed = 28
        // Değişiklik sürümü (CR-402): maxDaysAllowed = 14
        public static LoanPolicy defaultPolicy() {
            return new LoanPolicy(14); // Şartname güncellendi, kod yenilendi
        }
    }

    public static class LoanService {
        private final LoanPolicy policy;

        public LoanService(LoanPolicy policy) {
            this.policy = policy;
        }

        public int calculateDueOffset() {
            return policy.maxDaysAllowed();
        }
    }

    void main() {
        var service = new LoanService(LoanPolicy.defaultPolicy());
        System.out.println("✅ Şartname Değişikliği (CR-402) Koda Yansıdı: Yeni Vade = " +
service.calculateDueOffset() + " gün.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Acil bir bug çıktığında doğrudan koda girip 'quick fix' yapmayın! Önce şartnamedeki kuralı netleştirin, sonra /implement komutunu çalıştırın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never apply an unrecorded hotfix directly to production source code. Rectify the specification first, then execute generation to maintain single-truth traceability.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Yeni özellik, değişiklik veya bug fark etmez: Hepsi bir use case değişikliğidir.
- Şartnameyi atlayan her kod yaması, şartnamenin çürümesine ve projenin çökmesine yol açar.
- Değişiklik önce şartnamede onaylanır; ardından kod ve testler tek komutla yenilenir.

🎯 Key Architectural Takeaways

- Features, change requests, and bugs are all structurally a change to a use case.
- Any patch that bypasses the spec accelerates documentation rot and destroys regenerability.
- Changes are approved in the specification first; code and tests are regenerated deterministically.

50

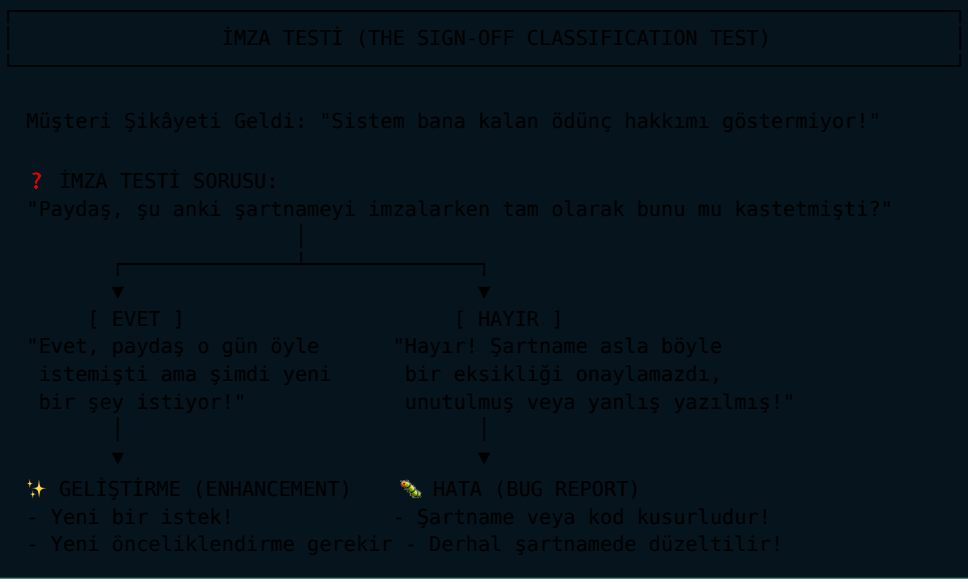
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

İmza Testi (Sign-Off Test): Hata (Bug) ile Geliştirme (Enhancement) Ayrımı

The Sign-Off Test: Distinguishing Genuine Bugs from Scope Enhancements

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir marangoza masa yaptırdınız. Masayı teslim alırken 'Ahşap ayaklı 4 kişilik meşe masa' sözleşmesini imzaladınız. 2 ay sonra 'Ben bu masanın mermer olmasını istiyordum' derseniz marangoza kızamazsınız; bu yeni bir istektir (Enhancement)! Ama masanın bir bacağı kısaysa ve sallanıyorsa bu bir hatadır (Bug)! İmza testi, sözleşmeye bakıp kimin haklı olduğunu saniyeler içinde anlar!

🇬🇧 ENGLISH (ELI5)

You commission a carpenter to build a dining table and sign a contract for 'Solid Oak Table, 4 seats'. Two months later you declare: 'I actually wanted white Italian marble!' The carpenter didn't make a mistake; that is an Enhancement request! But if the table arrived with 3 legs and wobbles, that is a Bug! The sign-off test inspects original intent to classify the discrepancy instantly.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İmza Testinin (The Sign-Off Test) Hukuki ve Mimari Gücü: Kurumsal yazılım geliştirmede geliştiriciler ile ürün sahipleri arasındaki en büyük sürtünme 'Bu bir bug mı yoksa yeni bir geliştirme mi?' tartışmasıdır. Hatalar garanti kapsamında bedelsiz ve derhal çözülürken, geliştirmeler yeni bütçe ve sprint planlaması gerektirir. Martinelli bu karmaşayı tek bir zarif soruyla çözer: 'Bir paydaş, şu an ne istediğini biliyorken, eski şartnameyi imzalar mıydı?' Eğer cevap 'Evet, o tarihte bunu istiyorduk ama şimdi fikrimiz değişti' ise, bu açıkça bir Değişiklik/Geliştirmedir (Enhancement). Eğer cevap 'Hayır, o şartname zaten bizim asıl niyetimizi yanlış ifade etmiş veya eksik bırakmış' ise, bu tartışmasız bir Hatadır (Bug). Bu test sınıflandırmayı duygu ve polemikten çıkarıp objektif bir sözleşme analizine dönüştürür.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Sign-Off Classification Test: The eternal debate between product owners and engineering leads centers on classifying discrepancies as either defects or scope changes. Defects typically carry contractual SLA penalties and zero additional billing, whereas scope changes require backlog refinement. Martinelli resolves this dispute via the Sign-Off Test: 'Would a stakeholder have signed off the old specification, knowing what they want now?' If yes, the specification captured true historical consensus, and the customer has since evolved their requirements. That is an Enhancement. If no, the specification either misstated stakeholder intent or omitted a crucial invariant. That is a Bug. This test converts an adversarial dispute into an objective specification reconciliation.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: İmza Testine Göre İş Akışının Otomatik Yönlendirilmesi
package com.maang.aiup.signoff;

public class SignOffClassificationDemo {
    public enum WorkType { BUG, ENHANCEMENT }

    public record DiscrepancyReport(String description, boolean wouldStakeholderHaveSignedOldSpec) {
        public WorkType classify() {
            // İmza Testi Mantığı
            return wouldStakeholderHaveSignedOldSpec ? WorkType.ENHANCEMENT : WorkType.BUG;
        }
    }

    void main() {
        var report1 = new DiscrepancyReport("Üye ödünç geçmişinde kitap ISBN'i görünmüyor", true);
        var report2 = new DiscrepancyReport("Ödünç limiti 5 olmasına rağmen sistem 6. kitaba izin
veriyor", false);

        System.out.println("Rapor 1 Sınıflandırması: " + report1.classify() + " (Yeni istek)");
        System.out.println("Rapor 2 Sınıflandırması: " + report2.classify() + " (Kritik iş kuralı
hatası)");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Müşteri 'bu bir bug' dedi diye şartnameye bakmadan hemen koda müdahale etmeyin; imza testini uygulayın, işin geliştirme mi hata mı olduğunu tespit edin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not rush to patch code simply because a client labels an issue as a 'critical bug'. Apply the sign-off test to classify intent accurately before touching specifications.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- İmza testi: Paydaş şu anki aklıyla eski şartnameyi imzalar mıydı?
- Cevap 'Evet' ise bu yeni bir geliştirmedir (Enhancement); 'Hayır' ise bir hatadır (Bug).
- Sınıflandırma testi mühendisler ile ürün sahipleri arasındaki polemiği tamamen bitirir.

🎯 Key Architectural Takeaways

- The sign-off test: Would a stakeholder have signed off the old spec, knowing what they want now?
- If yes, it represents an enhancement; if no, it is a genuine defect.
- The test eliminates subjective arguments between engineering and product management.

51

BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Tek Bir Döngü, Özel Komut Yok: Neden /bug Komutu Bulunmaz?

One Loop, No Special Commands: Why There is No /bug Command in AIUP

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                NEDEN /BUG VEYA /CHANGE KOMUTU YOKTUR?

Geleneksel Yanılgı:
- "Yeni özellik için /create-feature çalıştır!"
- "Bug için /bug-fix çalıştır!"
- "Değişiklik için /change-request çalıştır!"

❌ BU NEDEN YANLIŞTIR?:
Çünkü sistemin kod üretim motoru (/implement) ne tür bir iş yaptığınızı
umursamaz! Motor sadece GÜNCEL ŞARTNAMEYİ ve TESTLERİ okur!

✅ AIUP TEK DÖNGÜSÜ (ONE LOOP TO RULE THEM ALL):
┌──────────────────────────────────────────────────────────┐
│ 1. Niyet Belirleme: İmza testiyle durumu analiz edin.    │
│ 2. Şartnameyi Değiştir: UC-XXX dosyasında kuralı veya akışı güncelleyin! │
│ 3. Aynı Komutu Koş: `/implement UC-XXX`                  │
│ 4. Testler Güvenceye Alır: Değişen testler güncellenir, eski testler │
│    yeşil kalmaya devam eder!                             │
└──────────────────────────────────────────────────────────┘

🎯 NETİCE: Özel bir sihirli komut yoktur; şartname değişir, kod hizalanır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir 3D yazıcıyı düşünün: 3D yazıcıya 'Bu bir heykel', 'Bu kırık bir dişli tamiri' veya 'Bu yeni bir vazo' diye ayrı komutlar vermezsiniz. 3D yazıcıya sadece güncel 3D çizim dosyasını (CAD / Şartname) verirsiniz ve 'Yazdır' (Print / Implement) dersiniz! Yazıcı dosyadaki farkı kendisi basar. AIUP'ta da tek komut 'Hizala ve Üret'tir!

🇬🇧 ENGLISH (ELI5)

Consider an industrial 3D printer: You don't possess separate buttons for 'Print art', 'Print replacement gear', or 'Print vase'. You simply feed it the updated 3D CAD blueprint (The Specification) and press 'Print' (/implement)! The printer materializes the exact delta. In AIUP, there is only one universal generation loop.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Tek Bir Döngü İlkesi (One Loop, No Special Commands): Birçok AI destekli araç 'Fix Bug', 'Refactor', 'Add Feature' gibi onlarca farklı özel prompt veya komut icat etmeye çalışır. Simon Martinelli bu yaklaşımı reddeder: AI Unified Process'te ne bir `/bug` komutu ne de bir `/change-request` komutu vardır! İşin türü hangi komutu çalıştıracağınıza değil, şartnamenin neresini düzenleyeceğinize karar verir. 1) Sınıflandırma yapılır. 2) Gereksinim mühendisi şartnameyi günceller (örneğin kural metnini veya alternatif akış adımını düzeltir). 3) Her zamanki `/implement UC-XXX` komutu yeniden çalıştırılır. Üretim motoru mevcut Java sınıflarını (Controller, Entity, Repository, Test) okur ve şartnamedeki değişikliği yerinde uzlaştırır (reconciliation). 4) Testler güncellenen beklentiyi karşılar; kaldırılan senaryoların testleri silinir. Bu sayede hata çözümü geçici bir yama üretmek yerine, sistemin en değerli varlığı olan şartnameyi zenginleştiren kalıcı bir yatırıma dönüşür.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Single Uniform Loop Principle: Fragmented tooling often introduces divergent commands for defects versus enhancements (`/fix-bug`, `/patch`, `/refactor`). Martinelli explicitly rejects this fragmentation: there is no `/bug` or `/change-request` command in the AI Unified Process. The nature of the work dictates what you edit in the specification, never which CLI command you invoke. 1) Classify by intent via the sign-off test. 2) Update the specification first (correcting a business rule or refining a flow trigger). 3) Rerun the identical `/implement UC-XXX` command. The generator parses existing source files, inspects Git diffs, and reconciles implementations in place without duplicate scaffolding. 4) Tests carry the delta. Existing invariants remain green while modified assertions guard the new behavior. Fixing a defect becomes an asset enhancement rather than an unrecorded runtime patch.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Yerinde Uzlaştırma (In-Place Reconciliation) Mantığı
package com.maang.aiup.reconcile;

public class SpecReconciliationDemo {
    // Şartname: "Sistem parolanın en az 12 karakter olduğunu doğrular (BR-009)"
    // Eski Kod: 8 karakterdi (Hata tespit edildi ve şartnamede 12 olarak düzeltildi)
    public static class PasswordSecurityPolicy {
        public static boolean isPasswordStrong(String password) {
            // /implement komutu şartnamedeki BR-009'u okur ve sabiti 12 yapar:
            int minimumLength = 12; // Reconciled from spec!
            return password != null && password.length() >= minimumLength;
        }
    }

    void main() {
        System.out.println("8 Karakterli Şifre Geçerli mi?: " +
PasswordSecurityPolicy.isPasswordStrong("Pass1234"));
        System.out.println("14 Karakterli Şifre Geçerli mi?: " +
PasswordSecurityPolicy.isPasswordStrong("SuperStrongPass2026!"));
        System.out.println("✅ /implement şartnameyi okudu ve kodu yerinde güncelledi.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Hata düzeltirken test dosyasındaki assertion'ı doğrudan değiştirmeyin; assertion şartnameden türer. Önce use case şartnamesini değiştirin, testi şartname güncellesin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never alter an automated assertion directly during a bug fix; the assertion derives from the spec. Rectify the specification first, then reconcile the test suite.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- AIUP'ta /bug veya /patch komutu yoktur; tek bir evrensel üretim döngüsü vardır.
- İşin türü çalıştırılan komutu değil, şartnamede düzenlenen alanı belirler.
- Hata çözümleri şartnameye işlendiği için aynı hatanın tekrar oluşması imkansızdır.

🎯 Key Architectural Takeaways

- There is no /bug command in AIUP; all changes traverse the single specification loop.
- Work classification dictates which spec section to edit, not which command to run.
- Defects fixed through specs permanently inoculate the codebase against regressions.

52

BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Mevcut Sistemleri Kurtarma (Brownfield): Kodu Değil, Niyeti Kurtarın!

Brownfield Recovery: Reverse-Engineering Intent, Not Transcribing Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Toprağın altından kırık bir antik vazo çıkaran bir arkeolog düşünün: Arkeolog vazonun üstündeki çamuru ve tozu kopyalayıp müzeye koymaz! Çamuru dikkatlice temizler ve altındaki paha biçilmez altın deseni (Niyeti) ortaya çıkarır. Brownfield recovery, 10 yıllık eski kodun üstündeki SOAP ve XML çamurlarını temizleyip saf iş mantığını kurtarmaktır!

🇬🇧 ENGLISH (ELI5)

Think of an archaeologist excavating an ancient vase: The archaeologist doesn't duplicate the dirt and mud crust into museum showcases! They delicately brush off the grime to reveal the exquisite gold inlay beneath (The Intent). Brownfield recovery washes away 10 years of legacy XML and SOAP grime to rescue pure business logic!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Mevcut (Brownfield) Sistemlerde Tersine Mühendisliğin Felsefesi: Dünyadaki en kritik yazılımların %90'ı sıfırdan yazılmaz; yıllardır çalışan brownfield sistemlerdir. Martinelli'nin 'reverse-engineer' becerisi şu manifesto ile başlar: 'Siz kodu transkribe etmiyorsunuz. Siz o kodun tatmin etmek için inşa edildiği İŞ NİYETİNİ kurtarıyorsunuz ve bunu bir iş analistinin kod yazılmadan önce yazacağı seviyede kâğıda döküyorsunuz!' Concretely: 1) Controller ve View'lar use case değildir; use case'in kanıtlarıdır. 2) Koddaki her bir `if (x > y)` kontrolü muhtemel bir iş kuralı (BR-XXX) adayıdır. 3) Veritabanı şeması doğrudan Entity Model'e dönüştürülür. 4) Eğer bir use case adımı yalnızca o kodu okumuş birinin anlayacağı terimler içeriyorsa derhal silinmeli ve iş diline çevrilmelidir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Philosophy of Brownfield Reverse-Engineering: Greenfield green-pasture projects are rare; enterprise software is overwhelmingly brownfield. The AIUP reverse-engineering discipline is guided by an uncompromising stance: 'You are not transcribing the code. You are recovering the intent the code was built to satisfy, and writing it down at the level a business analyst would have written it before implementation.' Concretely: 1) Endpoints, controllers, and screens are not use cases; they are circumstantial evidence of use cases. 2) Guard clauses and validation branches are candidate business rules (BR-XXX), requiring translation into domain vernacular. 3) Database tables are evidence of entity model entities. 4) If a recovered step only makes sense to an engineer who has examined the repository source, it violates Zone 3 and must be rewritten into observable stakeholder behavior.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Legacy Spagettiden Kurtarılan Saf İş Niyeti
package com.maang.aiup.brownfield;

public class LegacyRecoveryDemo {
    // ❌ ESKİ MİRAS KOD (10 Yıllık Spagetti):
    // public void proc_99(int t, double a) { if (t == 4 && a > 500) ... }

    // ✅ KURTARILAN İŞ MODELİ (Clean & Expressive):
    public record CorporateOrder(String companyId, double amount, boolean hasDiscount) {}

    public static class RecoveredCorporateOrderPolicy {
        public static boolean isEligibleForVipProcessing(CorporateOrder order) {
            // Koddaki 't == 4' ifadesinin kurumsal müşteri olduğunu kurtardık!
            return order.amount() > 500.0 && order.hasDiscount();
        }
    }

    void main() {
        var order = new CorporateOrder("CORP-101", 750.0, true);
        System.out.println("✅ Miras Koddan Kurtarılan İş Kuralı Sonucu: "
            + RecoveredCorporateOrderPolicy.isEligibleForVipProcessing(order));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Eski sistemi kurtarırken 'hazır görmüşken bu hatayı da şartnamede düzelteyim' demeyin! Kurtarma (Discovery) ile Karar Verme (Deciding) iki ayrı aşamadır. Önce ne olduğunu yazın, sonra incelemede düzeltin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not attempt to fix business bugs while reverse-engineering legacy code. Recovery is discovery; decision-making is a subsequent stakeholder activity. Document what exists first.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Miras sistemlerde kod transkribe edilmez; kodun çözdüğü iş niyeti kurtarılır.
- Koddaki if-else guard blokları şartnamede BR-XXX iş kurallarına dönüştürülür.
- Kurtarma (keşif) ile karar verme (düzeltme) süreçleri birbirinden kesin olarak ayrılmalıdır.

🎯 Key Architectural Takeaways

- In brownfield systems, do not transcribe code; recover the underlying business intent.
- Guard clauses and validations in legacy code translate into candidate BR-XXX business rules.
- Discovery must remain separate from decision-making during legacy specification recovery.

53

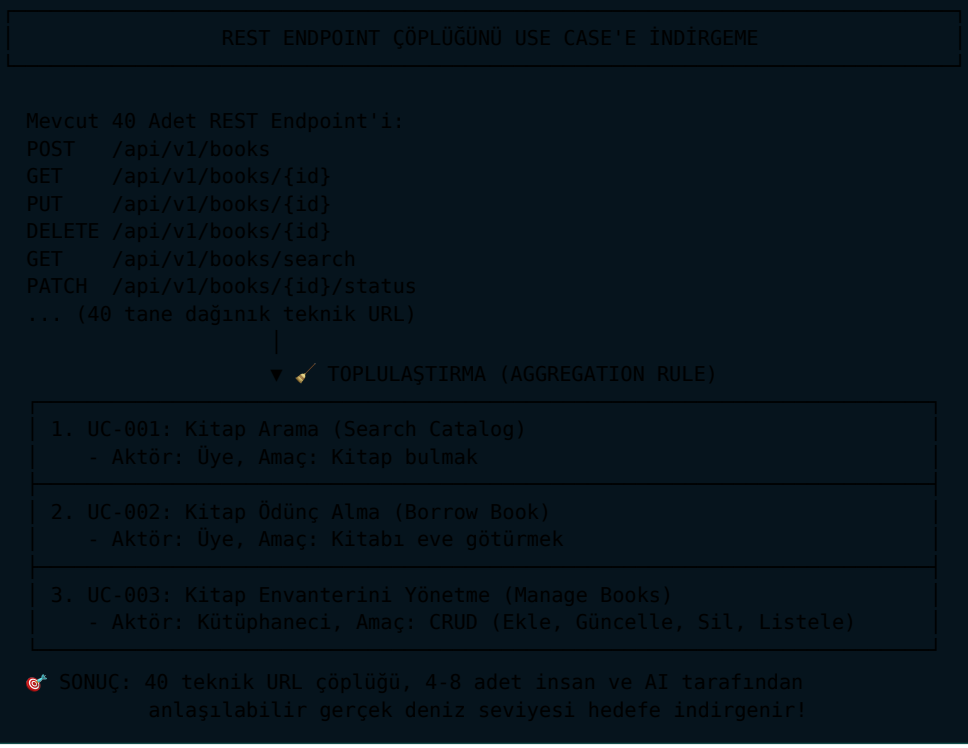
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Uç Noktaları Birleştirme: 40 REST Endpoint'ini 4-8 Use Case'e İndirgeme

Aggregating Endpoints: Collapsing 40 REST Endpoints into 4-8 Use Cases

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir akıllı telefonun içini açarsanız 500 tane lehimli kablo ve devre görürsünüz. Ama ekranı açtığınızda sadece 4 ikon görürsünüz: Telefon, Mesajlar, Kamera, Ayarlar! Kimse lehim kablosuyla uğraşmak istemez. İşte 40 REST endpoint'i o lehim kablolarıdır; 4-8 use case ise ekrandaki sade ve kullanışlı uygulama ikonlarıdır!

🇬🇧 ENGLISH (ELI5)

If you pry open an iPhone, you find 500 soldered micro-circuits and copper ribbons. But on the screen, you only interact with 4 icons: Phone, Messages, Camera, Settings! Nobody wants to touch raw solder. Forty REST endpoints are internal wiring; 4 to 8 use cases are the clean, intuitive app icons on the home screen!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Endpoint Toplulaştırma ve 'Manage X' Kuralı: Brownfield projelerde yapılan en yaygın hata, her bir Controller metodunu veya REST endpoint'ini bir use case zannetmektir. Bir projede 40 endpoint varsa, 40 tane use case yazmaya kalkan bir ekip şartname çöplüğünde boğulur. Martinelli'nin altın kuralı: 'Controller'lar, rotalar ve ekranlar use case değildir; use case'in delilleridir. Onları toplulaştırın!' 5 farklı CRUD endpoint'i (Create, Read, Update, Delete, List) aynı aktöre ve aynı veri akışına hizmet ettiği sürece tek bir 'Manage Books' use case'idir. Ancak ne zaman bir işlem kendi bağımsız iş kurallarına ve güvenlik rollerine sahip olursa (örn: 'Block Customer'), o zaman o işlem 'Manage X' çatısından ayrılıp kendi eylem seviyesi (action-level) use case'ine dönüşür. Tipik bir mikroservis bu yöntemle 4 ila 8 arası derli toplu use case'e indirgenir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Endpoint Aggregation and the 'Manage X' Heuristic: The fatal pitfall in legacy recovery is mistaking technical controllers or routes for use cases. Documenting 40 use cases for 40 HTTP endpoints yields bureaucratic paralysis. Martinelli's rule: 'Controllers, routes, and views are evidence of use cases, not use cases. Aggregate them.' Five REST endpoints (`POST /books`, `PUT /books/{id}`, `GET /books`, `DELETE /books/{id}`) are consolidated into a single 'Manage Books' use case so long as they share an identical actor and shape. Only when a sub-operation acquires unique preconditions, distinct role-guards, or diverging state transitions (e.g., 'Archive Book' requiring audit approval) does it split out as an action-level use case. A production microservice typically condenses into 4 to 8 user-goal specifications.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 4 REST Endpoint'inin Tek Bir 'ManageBooks' Servisinde Toplulaştırılması
package com.maang.aiup.aggregation;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class EndpointAggregationDemo {
    public record BookRecord(String id, String title, String author) {}

    // Tek bir 'UC-003: Manage Books' use case'i altındaki CRUD operasyonları
    public static class ManageBooksUseCase {
        private final Map<String, BookRecord> db = new ConcurrentHashMap<>();

        public void createBook(BookRecord book) { db.put(book.id(), book); }
        public BookRecord readBook(String id) { return db.get(id); }
        public void updateBook(BookRecord book) { db.put(book.id(), book); }
        public void deleteBook(String id) { db.remove(id); }
    }

    void main() {
        var uc = new ManageBooksUseCase();
        uc.createBook(new BookRecord("B1", "Effective Java", "Joshua Bloch"));
        System.out.println("✅ 4 REST endpoint'i tek bir 'Manage Books' use case'inde
toplulaştırıldı.");
        System.out.println("   -> Okunan Kitap: " + uc.readBook("B1"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Her HTTP GET veya POST isteği için ayrı bir use case dosyası açmayın! CRUD işlemlerini tek bir 'Manage X' altında toplayın; yalnızca kuralları ayrışan işlemleri split edin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not generate individual use case documents for every HTTP GET or POST route. Aggregate standard CRUD into 'Manage X'; split only when business policies diverge.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- REST uçları use case değildir; use case'in varlığına işaret eden teknik izlerdir.
- Aynı aktör ve akışı paylaşan CRUD işlemleri tek bir 'Manage X' altında toplanır.
- Tipik bir kurumsal mikroservis 40 endpoint'ten 4-8 anlamlı use case'e indirgenir.

🎯 Key Architectural Takeaways

- REST endpoints are not use cases; they are technical evidence of underlying use cases.
- CRUD operations sharing the same actor and flow aggregate into a single 'Manage X' spec.
- A typical microservice condenses from 40 low-level routes into 4 to 8 meaningful use cases.

54

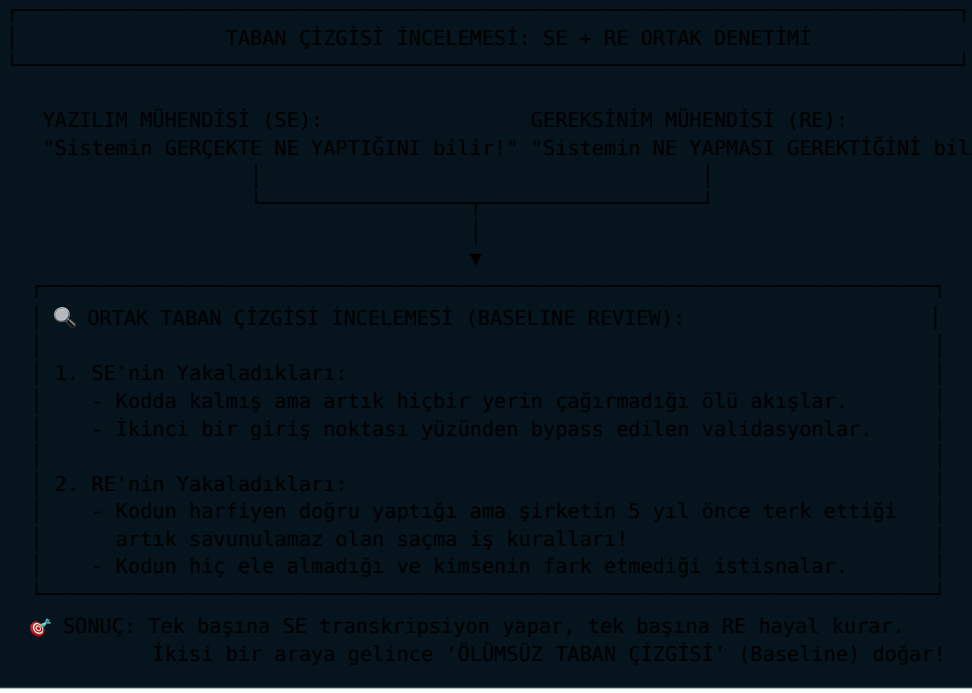
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Taban Çizgisi İncelemesi (Baseline Review): SE ve RE Ortak Eşleşmesi

The Baseline Review: Software Engineer & Requirements Engineer Joint Pairing

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski bir evi tadilata sokacağınızı düşünün: Tesisat ustası (Yazılım Mühendisi) der ki: 'Bu boru patlak, bu kablo hiçbir yere bağlanmıyor!' İç mimar (Gereksinim Mühendisi) der ki: 'Bu duvarı yıkıp salonu büyütmemiz lazım, o oda artık kullanılmıyor!' Tesisatçı tek başına evi anlayamaz, mimar tek başına boruları bilemez. İkisi el ele verince ev saraya dönüşür!

🇬🇧 ENGLISH (ELI5)

Imagine renovating a 100-year-old mansion: The plumber (Software Engineer) notes: 'This pipe is rusted, and this wire leads nowhere!' The interior architect (Requirements Engineer) says: 'We need to knock down this wall to expand the living room; that pantry is obsolete!' Neither can restore the estate alone. Together, they create a master renovation baseline!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Taban Çizgisi İncelemesinin (The Baseline Review) Kritik Rolü: Mevcut bir sistemden çıkarılan tersine mühendislik taslakları, koda bakarak yazıldığı için süreçteki 'en az güvenilir' belgelerdir. Bu taslakları güvenilir bir 'Taban Çizgisi' (Spec Baseline) haline getiren şey, iki farklı disiplinden uzmanın bir odaya kapanıp yaptığı ortak incelemedir: 1) Yazılım Mühendisi (SE): Kodun gerçekte ne yaptığını bilir. Artık hiçbir ekranın tetiklemediği ölü kod yollarını, arka kapılardan bypass edilen güvenlik açıklarını ve eski bir sabitten türetilmiş gereksiz kuralları yakalar. 2) Gereksinim Mühendisi (RE): Sistemin ne yapması gerektiğini veya kime soracağını bilir. Kodun harfiyen doğru icra ettiği ama günümüz iş mantığına tamamen aykırı düşen mantıksız akışları, kodun hiç yakalamadığı istisnaları ve şirketin 5 yıl önce bıraktığı terimleri temizler. İnceleme sonucunda uzlaşılamayan her konu (belgelenmemiş ama kullanılan davranışlar, bozulmuş fonksiyonlar) bir Değişiklik Günlüğü'ne (Change Log) yazılır ve paydaş onayına sunulur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Baseline Review Pairing: A recovered specification draft written from source code inspection is inherently untrustworthy—it describes what was compiled, not what was intended. Elevating these drafts into an authoritative 'Spec Baseline' requires a rigorous joint pairing between two complementary roles: 1) The Software Engineer (SE) understands actual system behavior. They catch obsolete paths nothing invokes, validations bypassed by secondary routes, and fake business rules inferred from legacy constants. 2) The Requirements Engineer (RE) understands business domain intent. They catch flows that are mathematically accurate to the code but indefensible to customers, unhandled failure modes that went unnoticed for years, and deprecated terminology. Neither role can succeed in isolation: an SE alone produces formatted code transcription; an RE alone produces an ungrounded wish list. The reconciled output establishes the spec baseline from which all future development proceeds.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Baseline Review Sonrası Temizlenen Taban Çizgisi Modeli
package com.maang.aiup.baseline;

public class BaselineReviewReconciliation {
    // SE ve RE İncelemesinden Geçmiş Uzlaşılmış Varlık
    public record ReconciledLoanRule(
        boolean deadCodeEliminated,
        boolean businessIntentVerified,
        String approvedPolicy
    ) {}

    public static class BaselineReviewAuditor {
        public static ReconciledLoanRule audit(boolean seApproved, boolean reApproved) {
            if (seApproved && reApproved) {
                return new ReconciledLoanRule(true, true, "✅ APPROVED BASELINE: Vade 28 gün, Maksimum
5 kitap.");
            }
            throw new IllegalStateException("❌ İnceleme tamamlanmadı! Her iki uzmanın onayı
şarttır.");
        }
    }

    void main() {
        var baseline = BaselineReviewAuditor.audit(true, true);
        System.out.println("Taban Çizgisi İnceleme Sonucu: " + baseline.approvedPolicy());
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Taban çizgisi incelemesini tek bir uzmana (sadece yazılımcıya veya sadece analiste) emanet etmeyin; yazılımcı koda aşık olur, analist ise gerçeklerden kopar.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never assign the baseline review to a single person. An engineer transcribes code habits; an analyst invents an ungrounded wish list. Pairing is indispensable.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Kurtarılan taslak şartnameler, SE ve RE ortak incelemesinden geçerek 'Baseline' olur.
- Yazılım mühendisi kod gerçekliğini, gereksinim mühendisi ise iş hedeflerini korur.
- Uzlaşılamayan davranışlar doğrudan Değişiklik Günlüğü'ne (Change Log) aktarılır.

🎯 Key Architectural Takeaways

- Recovered specification drafts become an authoritative 'Baseline' via joint SE-RE review.
- The software engineer guards code truth; the requirements engineer guards business intent.
- Unresolved ambiguities populate the initial change log for formal stakeholder arbitration.